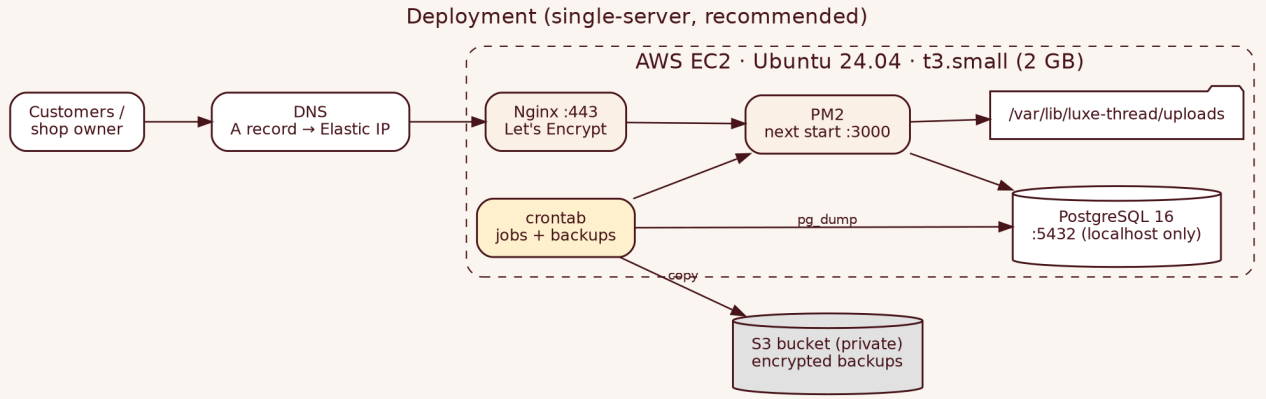
LUXE & THREAD · DOCUMENTATION

INTERNAL – for developers

Internal 15 · Deployment Guide (AWS EC2 + PostgreSQL)

Audience	INTERNAL – DevOps
Recommended	one Linux server (EC2 `t3.small`) running Nginx + Node (PM2) + PostgreSQL
Alternatives	Vercel for the app + managed Postgres (Neon/Supabase/RDS). **Uploads** then need object storage (S3) – the code writes to `UPLOAD_DIR`, so on Vercel that part must be adapted first



1 · Create the server

1. EC2 → Launch: Ubuntu 24.04 LTS, `t3.small`, 30 GB gp3. 2. Security group: SSH (22) **from your IP only**, 80, 443 open. 3. Elastic IP. 4. DNS `A` records (`@`, `www`) → Elastic IP.

2 · Install software

```
ssh -i key.pem ubuntu@<elastic-ip>
sudo apt update && sudo apt -y upgrade
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -        # adds Node 22 package source
sudo apt -y install nodejs nginx git ufw postgresql postgresql-client awscli
sudo npm i -g pm2                                                        # process manager
sudo ufw allow OpenSSH && sudo ufw allow 'Nginx Full' && sudo ufw --force enable
```

3 · Database

```
sudo -u postgres psql <<'SQL'
CREATE USER luxe WITH PASSWORD 'CHANGE-ME-LONG-RANDOM';
CREATE DATABASE luxe_thread OWNER luxe;
SQL
# DATABASE_URL=postgres://luxe:CHANGE-ME@localhost:5432/luxe_thread
```

PostgreSQL listens on localhost by default – keep it that way.

4 · Application

```
git clone <repo-url> fashion-store && cd fashion-store && git checkout v2.0.0
sudo mkdir -p /var/lib/luxe-thread/uploads && sudo chown -R ubuntu:ubuntu /var/lib/luxe-
thread
cp .env.example .env.production.local && chmod 600 .env.production.local && nano
.env.production.local
#   set: NEXT_PUBLIC_SITE_URL, DATABASE_URL, SESSION_SECRET, JOBS_SECRET,
UPLOAD_DIR=/var/lib/luxe-thread/uploads, live Razorpay/Shiprocket/SMTP values
npm ci
npm run db:migrate                      # creates tables   (do NOT run db:seed in
production)
npm run admin:create -- owner@your-domain.in "Strong Passphrase 123" "Owner" owner
npm run build                           # needs internet once (Google fonts)
pm2 start npm --name luxe-thread -- start
pm2 save && pm2 startup                 # run the printed command so it restarts after
reboot
```

5 · Nginx + HTTPS

```
server {
  server_name your-domain.in www.your-domain.in;
  client_max_body_size 30m;                      # CSV and photo uploads
  location / {
    proxy_pass http://127.0.0.1:3000; proxy_http_version 1.1;
    proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-
Forwarded-Proto $scheme;
  }
}
```

```
sudo ln -s /etc/nginx/sites-available/luxe-thread /etc/nginx/sites-enabled/ && sudo nginx -t
&& sudo systemctl reload nginx
sudo apt -y install certbot python3-certbot-nginx && sudo certbot --nginx -d your-domain.in
-d www.your-domain.in
```

`X-Forwarded-For` matters: rate limiting uses the real visitor IP. Photo/CSV uploads go through Server Actions, whose default body limit is 1 MB; `next.config.ts` raises it to **25 MB** (`experimental.serverActions.bodySizeLimit`). Keep Nginx `client_max_body_size` ≥ that value.

6 · Cron jobs (`crontab -e` as ubuntu)

```
* * * * *   curl -fsS -X POST -H "x-jobs-secret: <JOBS_SECRET>" https://www.your-domain.in/api/jobs/notifications  >/dev/null
*/5 * * * * curl -fsS -X POST -H "x-jobs-secret: <JOBS_SECRET>" https://www.your-domain.in/api/jobs/expire-orders >/dev/null
```

(and the backup in Internal 10).

7 · Register webhooks, then run the health check

Razorpay and Shiprocket webhooks per Internal 04; then Internal 08 §A.

8 · Update / rollback

See MOP §10. Always back up first.

9 · Sizing & scaling

`t3.small` handles a boutique comfortably. Scale path: bigger instance → managed Postgres (RDS) → second app server behind a load balancer (requires shared uploads on S3 and Redis rate-limit/session-independent design – cookies are stateless already).

10 · Vercel notes

Possible for the app, but: (1) serverless has no persistent disk → move photos to S3/Cloudinary; (2) use a managed Postgres with pooling (`DATABASE_URL` pooled); (3) cron via Vercel Cron calling the two job endpoints; (4) `after()` and long tasks have time limits – keep jobs small.